Algorithm – Part 2: the Growth of Functions

Shuai Zhang

Department of Computer Science
University of Central Oklahoma

Motivation

- There are many different algorithms solving similar questions, how to make an evaluation of them?
  - A commonly used metric: Time Complexity
    - For different input, the actual time used would be different
    - The scale/level of total time used can be estimated by a **function**
      - For example insertion sort works at n^2 level: two for loops over all the terms in the list
      - Typically loops will increase the scale/level
    - How to decide which algorithm is better?
      - Compare the time complexity function: $n^2 > n$
      - Smaller function means the algorithm is more efficient

Introduction: The Growth of Functions$_1$

- We care about the influence when the input size grows
  - Compare the efficiency of two algorithms
  - Determine whether it is practical to use an algorithm as the input grows

```
int n= 10000;     // input size
for (int i = 0; i < n; i++)
   for (int j = 0; j < n; j++)
      for (int k = 0; k < n; k++)  // what if just two loops?
         Perform an action
```

The action part would be executed one trillion times. If it takes one microsecond (10^{-6} second) to perform the action, the total time to run the loop would be more than 277 hours.

Introduction: The Growth of Functions$_2$

- For small problems, you can still use a bad algorithm to solve it
  - Since the input size is small, there won't be significant difference on the total execution time
- We care about the behavior for **large input**
  - Basic comparison is: $f_1(n) = 100$ v.s.$f_2(n) = n$
    - **Constant won't grow**!
    - A constant won't contribute much to "the growth of functions": the variables with big values would make significant difference
    - Large input means n could be very big, and its trend is to be bigger!
    - The growth rate is very similar to the derivative in Calculus.

Introduction: The Growth of Functions$_3$

- Those functions can be at different scales/levels
  - $f_1(n) = 2n^2$ and $f_2(n) = 3n^2$ are at the same level
    - They are both at n^2 level
    - Again **constant won't make big difference** in the growth rate
  - Levels could be represented as Big-O Notations
    - E.g. $O(1)$, $O(n^2)$, $O(n^3)$, $O(nlogn)$: each notation is a **representative** of each level
    - Still based on the idea: constant won't influence the level
      - In $O(1)$, it could be $f_1(n) = 100$ or $f_2(n) = 1000$, all constant level
      - In $O(n^2)$, it could be$f_1(n) = 100n^2$ or $f_2(n) = 1000n^2 + 10n$

Big-O Notation

- Focus on
  - Large input
  - Categories (levels) of functions: constant won't grow
- **Definition**: Let f and g be functions $\mathbb{Z} \to \mathbb{Z}$ or $\mathbb{R} \to \mathbb{R}$. We say that $f(x)$ is $O(g(x))$ if there are constants C and k such that

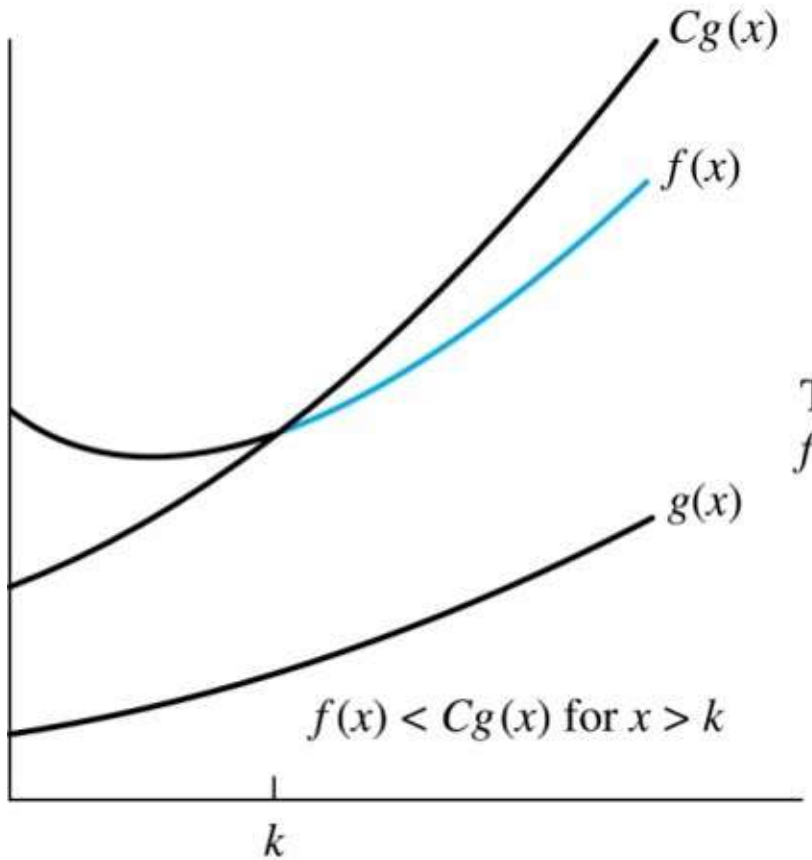
$$|f(x)| \leq C|g(x)|$$

when $x > k$. Constant k is used to define "large input"

Constant C is used to show they are at the same level

  - This is read as "$f(x)$ is big-O of $g(x)$" or "g asymptotically dominates f", or "
  - The constants and are called ***witness*** to the relationship "$f(x)$ is $O(g(x))$". Only one pair of witnesses is needed.

Illustration of Big-O Notation



The part of the graph of $f(x)$ that satisfies $f(x) < Cg(x)$ is shown in color.

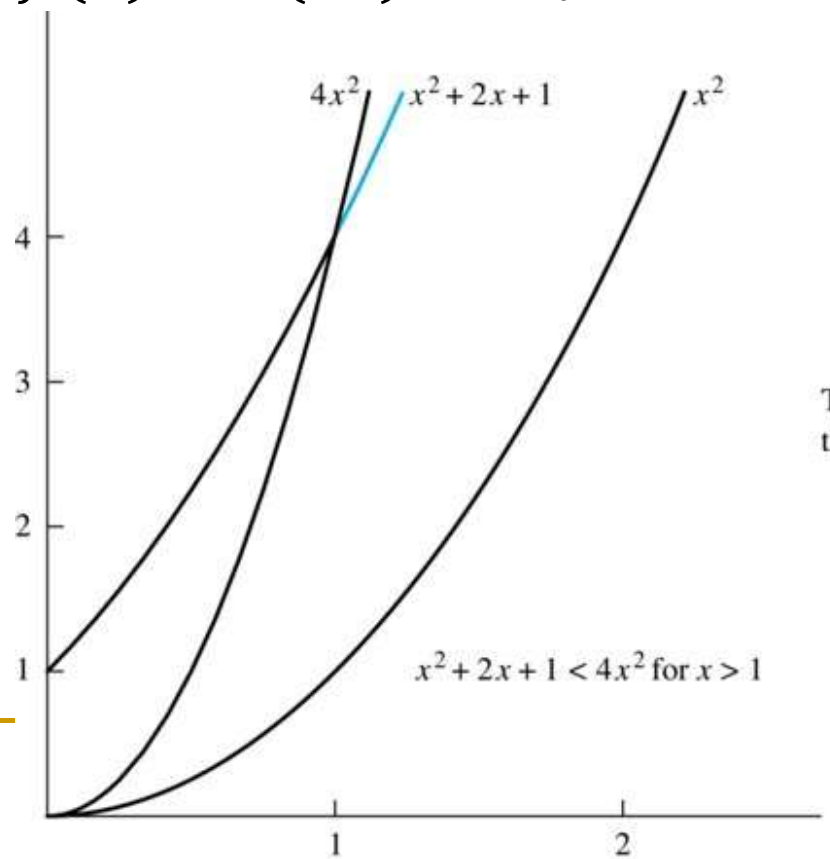
- After the intersection point, the second curve is between the first and the third curves as highlighted.

Example: Use of Big-O Notation

- **Example**: Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$.
- **Solution**: For $x > 1$, we have $x < x^2$ and $1 < x^2$, then

$$0 \le x^2 + 2x + 1 \le x^2 + 2x^2 + x^2 \le 4x^2$$

so $f(x)$ is $O(x^2)$ (with witness $C = 4$ and $k = 1$)



The part of the graph of $f(x) = x^2 + 2x + 1$ that satisfies $f(x) < 4x^2$ is shown in blue.

Notes$_1$

- We may see "$f(x) = O(g(x))$" instead of "$f(x)$ is $O(g(x))$".

- It's OK to write $f(x) \in O(g(x))$, because $\boldsymbol{O(g(x))}$ **represents the set of functions that are $\boldsymbol{O(g(x))}$.**
  - This set is called a **complexity class**.

- Usually, we don't need absolute sign in the function since in time complexity analysis we will always deal with functions that take on positive values (e.g. time used is always positive).

Notes$_2$

- Why it's called big-O notation: different Orders of growth rates
  - e.g. $O(1)$: at order 1
  - e.g. $O(n)$: ay order n
  - e.g. $O(n^2)$: at order n squared
- With big-O notation, we can estimate the growth of a function without worrying about constant multipliers or **smaller order terms**.
  - $f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$ is $O(x^n)$

Practices 1

- Justify the correctness of following statement:
  - $f(x) = x^2$ is $O(x^2 + 2x + 1)$: True or False?
  - $f(x) = 7x^2$ is $O(x^3)$: True or False?
  - $f(n) = n^2$ is $O(n)$: True or False?
  - $f(x) = x^3$ is $O(7x^2)$: True or False?

Practices 2

$g(x)$ is the Upper Bound of $f(x)$

- Justify the correctness of following statement:
  - $f(x) = x^2$ is $O(x^2 + 2x + 1)$: **True**
    For $x > 0$, $x^2 \leq x^2 + 2x + 1$ $\quad (C = 1, k = 0)$
  - $f(x) = 7x^2$ is $O(x^3)$: **True**
    For $x > 7$, $7x^2 \leq x^3$ $\quad (C = 1, k = 7)$
  - $f(n) = n^2$ is $O(n)$: **False**
    Assume true, then for $n > k$, $n^2 \leq$ Cn, then $n \leq C$, contradiction.
  - $f(x) = x^3$ is $O(7x^2)$: **False**
    Assume true, then for $x > k$, $x^3 \leq 7Cx^2$, then $x \leq 7C$, contradiction.
  - There could be a lot of witness pairs: you only need to find one pair.

Practices 3

- Prove the following statements:
  - $f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$ is $O(x^n)$
  - $f(n) = log\mathrm{n}$ is $O(n)$

Practices 4

- Prove the following statements:
  - $f(x) = a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0$ is $O(x^n)$

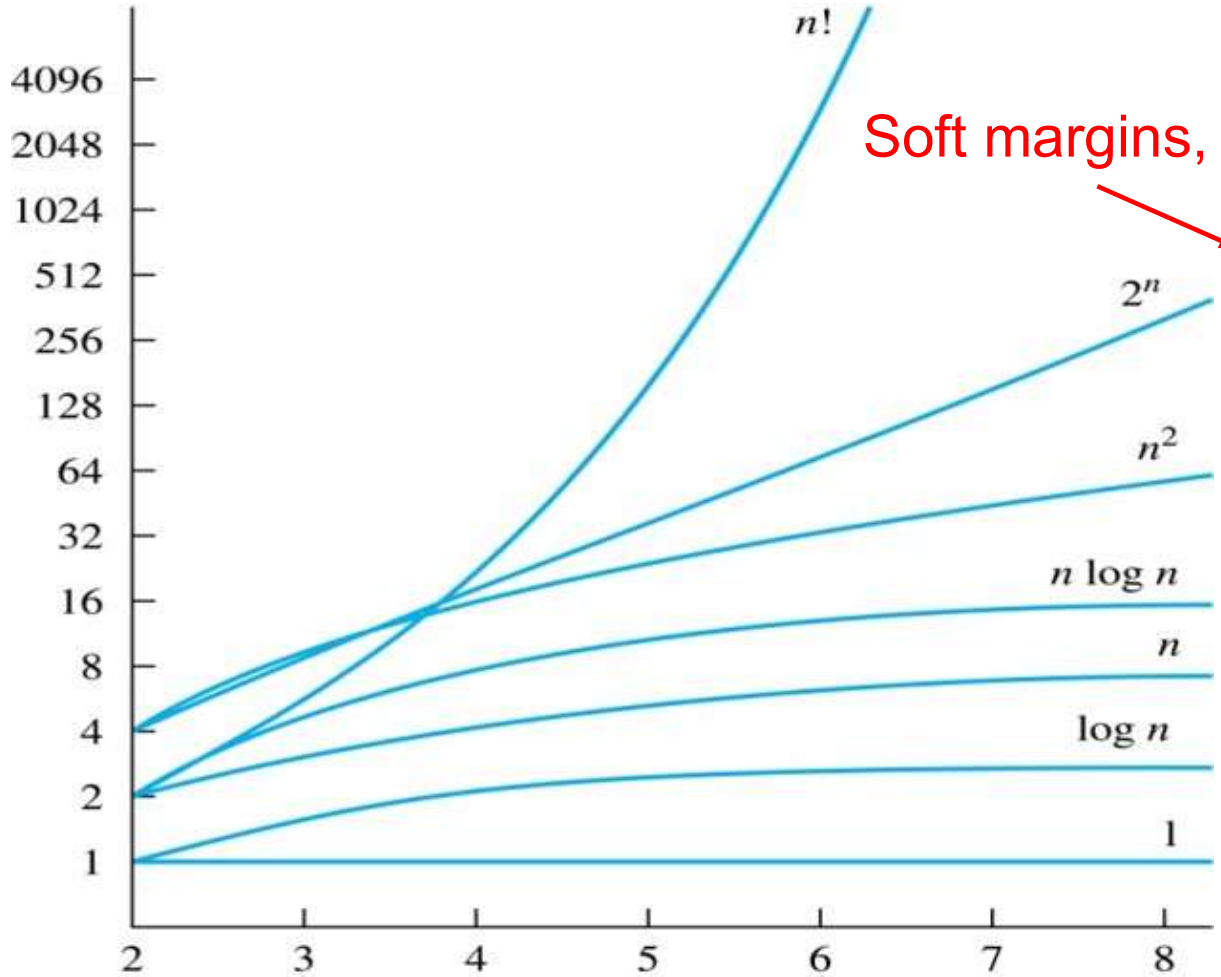
$$f(x) = a_n x^n + \frac{a_{n-1}x^n}{x} + \cdots + \frac{a_1 x^n}{x^{n-1}} + \frac{a_0 x^n}{x^n}$$
$$= x^n\left(a_n + \frac{a_{n-1}}{x} + \cdots \frac{a_1}{x^{n-1}} + \frac{a_0}{x^n}\right)$$
$$\leq x^n(a_n + a_{n-1} + \ldots + a_1 + a_0), \text{ when } x > 1$$

  - $f(n) = log\text{n}$ is $O(n)$

It's easy to have $n < 2^n$ when $n > 1$, then

$$log\text{n} < \log(2^n)$$
$$log n < n$$

Growth : Graphs of the functions



$O(n!)$

Soft margins, e.g. $2^n + 3n^2 + 5n$

$O(2^n)$

$O(n^2)$

$O(nlogn)$

$O(n)$

$O(logn)$

$O(1)$

- The behavior when n gets larger

A group of functions, e.g. $logn + 3$, $3logn$

Practices 5

- Find the least integer n such that f(x) is $O(x^n)$:
  - $f(x)=2x^3 + x^2\log x$
  - $f(x)=3x^4 + (\log x)^5$

Practices 6

- Find the least integer n such that f(x) is $O(x^n)$:
  - $f(x)=2x^3 + x^2\log x$
    - $f(x)=2x^3 + x^2\log x < 2x^3 + x^2 \cdot x = 3x^3$
      (because $\log x < x$ for large values of x)
    - f(x) is $O(x^3)$

  - $f(x)= 3x^4 + (\log x)^5$
    - $3x^4$ will dominate this expression
      - Let $m = \log x$, then $x = 2^m$, so $f(x) = 3\,(2^m)^4 + m^5$
      - Exponential dominates polynomial
    - f(x) is $O(x^4)$

Big-Omega Notation

- Big-O notation is used to set the upper bound, we can also define a lower bound in a similar way
- **Definition**: Let f and g be functions $\mathbb{Z} \to \mathbb{Z}$ or $\mathbb{R} \to \mathbb{R}$. We say that $f(x)$ is $\Omega(g(x))$ if there are constants C and k such that

$$|f(x)| \geq C|g(x)|$$

  when $x > k$.

  - This is read as "$f(x)$ is big-Omega of $g(x)$", "$f(x)$ is $\Omega(g(x))$".
  - Big-Omega tells us that a function grows at least as fast as another.
- Example: $f(x) = 8x^3 + 5x^2$ is $\Omega(x^3)$
  $g(x) = x^3$ is also $\Omega(8x^3 + 5x^2)$
- Think about: $f(x)$ is $\Omega(g(x))$ if and only if $g(x)$ is $O(f(x))$

Big-Theta Notation

- **Definition**: Let f and g be functions $\mathbb{Z} \to \mathbb{Z}$ or $\mathbb{R} \to \mathbb{R}$. The function $f(x)$ is $\Theta(g(x))$ if $f(x)$ is $O(g(x))$ and $f(x)$ is $\Omega(g(x))$.
  - We say that "$f(x)$ is big-Theta of $g(x)$" or "$f(x)$ is of order $g(x)$" or "$f(x)$ and $g(x)$ are of the same order"
  - $f(x)$ is $\Theta(g(x))$ if and only if there exists constants C_1, C_2 and k such that $C_1 g(x) \leq f(x) \leq C_2 g(x)$when $x > k$. This follows from the definitions of big-O and big-Omega.

Example: Use of Big-Theta Notation

- **Example**: Show that $f(x) = 3x^2 + 8xlogx$ is $\Theta(x^2)$
- **Solution**: when x>1,

$$0 \leq 8xlogx \leq 8x^2$$

$$3x^2 \leq 3x^2 + 8xlogx \leq 11x^2$$

so $f(x) = 3x^2 + 8xlogx$ is $\Theta(x^2)$

- Or we can easily tell $f(x)$ is $O(x^2)$ and $f(x)$ is $\Omega(x^2)$.

Extension 1

- What's the big-O of $\frac{3x^4+2}{5x^3+3x^2}$?
  - $O(x)$

Extension 2

- When $n > 2$, we have

$$1 \leq logn \leq n \leq nlogn \leq n^2 \leq 2^n$$

  - The right-side function is **upper bound** of left-side function
  - $logn$ is $O(logn)$, $O(n)$, $O(nlogn)$ … …
  - $O(1) \subseteq O(logn) \subseteq O(n) \subseteq O(nlogn) \subseteq O(n^2) \subseteq O(2^n)$
- If f is $O(g)$, then $O(f) \subseteq \mathrm{O(g)}$
  - If f is $O(g)$ and g is $O(f)$, then $O(f) = \mathrm{O(g)}$

Extension 3

- Big-O notations on **combinations of functions**
  - If $f_1(x)$ and $f_2(x)$ are both $O(g(x))$, then $(f_1 + f_2)(x) = f_1(x) + f_2(x)$ is $O(g(x))$
    - Try $f_1(x) = x + 1$ and $f_2(x) = 2x^2 + 3x$
  - If $f_1(x)$ is $O(g_1(x))$, and $f_2(x)$ is $O(g_2(x))$, then $(f_1 + f_2)(x) = f_1(x) + f_2(x)$ is $O(\max(g_1(x), g_2(x)))$ and $(f_1 f_2)(x) = f_1(x) f_2(x)$ is $O(g_1(x) g_2(x))$
    - Try $f_1(x) = x + 1$ and $f_2(x) = 2x^2 + 3x$

- Think about **composition functions**:
  - If $f(x)$ is $O(g(x))$, for any $h(x)$, could $h \circ f(x)$ be $O(h \circ g(x))$?
  - Try $g(n) = n$, $f(n) = 2n$, $h(n) = 2^n$ $(hint: 2^{2n} = 2^n * 2^n)$